

PetFinder

Requisitos do Sistema

Especificação completa dos requisitos funcionais e não funcionais do aplicativo de reconhecimento de cães e gatos perdidos via inteligência artificial.

Versão 1.0.0 Stack Flutter · FastAPI · CLIP · ChromaDB · Docker

Plataformas Android · iOS Tipo Portfólio acadêmico

ÍNDICE

– Visão geral	RF-01 Cadastro de pet perdido
RF-02 Busca por similaridade	RF-03 Listagem e consulta
RF-04 Processamento de IA	RF-05 Gerenciamento de imagens
RNF-01 Desempenho	RNF-02 Segurança e privacidade
RNF-03 Usabilidade	RNF-04 Infraestrutura
RNF-05 Qualidade de código	– Glossário

22

requisitos
totais

13

funcionais (RF)

9

não funcionais
(RNF)

5

módulos

1 Requisitos Funcionais

Descrevem o que o sistema deve fazer — comportamentos, funcionalidades e regras de negócio observáveis pelo usuário ou por outros sistemas.

RF-01.1

Formulário de cadastro de pet perdido

must have

O aplicativo deve disponibilizar um formulário completo para que o tutor registre um animal desaparecido. O formulário deve coletar: nome do pet, espécie (cão ou gato), raça, cor predominante, porte (pequeno, médio ou grande), característica diferencial (campo livre), telefone de contato, cidade onde o animal sumiu e data do desaparecimento.

Ator	Tutor do animal perdido
Pré-condição	App instalado e backend em execução
Pós-condição	Pet salvo no SQLite com ID gerado; embeddings salvos no ChromaDB
Validação	Campos obrigatórios: nome, espécie, cor, porte, telefone, cidade, data
Endpoint	POST /pets com multipart/form-data

RF-01.2

Upload de múltiplas fotos do pet

must have

Durante o cadastro, o tutor deve poder anexar uma ou mais fotos do animal, tanto da câmera do dispositivo quanto da galeria. Quanto mais fotos forem enviadas, mais precisos serão os embeddings gerados pela IA. O sistema deve aceitar formatos JPG, PNG e WebP.

Mínimo	1 foto obrigatória por cadastro
Recomendado	3 a 5 fotos de ângulos diferentes
Formatos	JPG, PNG, WebP
Fontes	Câmera nativa e galeria do dispositivo via <code>image_picker</code>

RF-01.3

Confirmação de cadastro com feedback visual

must have

Após o envio bem-sucedido, o aplicativo deve exibir uma mensagem de confirmação clara informando que o pet foi cadastrado. Durante o processamento — que pode levar alguns segundos enquanto a IA gera os embeddings — deve ser exibido um indicador de progresso com texto explicativo ("Processando IA...").

Sucesso	Dialog com ícone verde e botão de retorno à tela inicial
Erro	SnackBar vermelho com mensagem descritiva do problema
Loading	CircularProgressIndicator + texto "Processando IA..."

RF-02.1

Envio de foto do animal encontrado para análise

must have ▼

O usuário que encontrou um animal deve conseguir tirar uma foto diretamente pela câmera ou selecionar da galeria e enviá-la ao backend para análise. A foto é o principal insumo para a comparação por IA.

Ator	Pessoa que encontrou um animal
Endpoint	POST /search com multipart/form-data
Campos opcionais	Raça observada, cor, porte — melhoram a precisão

RF-02.2

Exibição de resultados com percentual de similaridade

must have ▼

O aplicativo deve exibir uma lista ordenada dos pets mais similares ao animal encontrado. Cada resultado deve mostrar: nome, foto, raça, cor, porte, cidade, data do desaparecimento, característica diferencial e o percentual de compatibilidade calculado pela IA (0% a 100%).

Ordenação	Maior similaridade primeiro
Limite padrão	5 resultados por busca (configurável)
Badge verde	Similaridade $\geq$ 70% — alta probabilidade
Badge amarelo	Similaridade 40%-69% — média probabilidade
Badge laranja	Similaridade < 40% — baixa probabilidade

RF-02.3

Informações de contato acessíveis a partir do resultado

must have ▼

Cada card de resultado deve exibir o telefone do tutor e um botão de contato. Ao tocar, o sistema deve exibir o número e, preferencialmente, abrir o discador nativo ou o WhatsApp (via `url_launcher`).

Implementação v1	SnackBar exibindo o número (mínimo funcional)
Implementação v2	Abertura do discador via <code>tel:</code> ou WhatsApp via <code>https://wa.me/</code>

RF-04.1

Geração de embeddings de imagem com CLIP

must have

Para cada foto de pet cadastrado, o sistema deve usar o modelo CLIP (clip-ViT-B-32 via sentence-transformers) para gerar um vetor numérico de alta dimensão (embedding) que representa o conteúdo visual da imagem. Esse vetor é o que permite a comparação matemática entre fotos.

Modelo	clip-ViT-B-32 — leve e preciso para portfólio
Biblioteca	sentence-transformers 2.7.0
Dimensão	512 dimensões por embedding
Armazenamento	ChromaDB com distância cosseno

RF-04.2

Geração de embeddings de texto para atributos do pet

must have

Além da imagem, o sistema deve gerar um embedding de texto a partir da descrição textual do pet (raça + cor + porte + diferencial) usando o mesmo modelo CLIP. Isso permite que características escritas também influenciem na comparação.

Exemplo de texto	"golden retriever dourado grande mancha olho esquerdo"
Peso no score final	30% texto + 70% imagem = 100%

RF-04.3

Cálculo de similaridade cosseno e score combinado

must have

Na busca, o sistema deve calcular a similaridade cosseno entre o embedding da foto enviada e todos os embeddings cadastrados no ChromaDB. O score final deve combinar imagem (70%) e texto (30%), resultando em um percentual de 0% a 100%. Quando um pet tem múltiplas fotos, o maior score entre elas é usado.

Fórmula	$\text{score} = 1 - \text{distância_cosseno} / 2 \times 100$
Agrupamento	Múltiplas fotos do mesmo pet → melhor score prevalece
Ajuste de pesos	Configurável via IMAGE_WEIGHT e TEXT_WEIGHT em config.py

2 Requisitos Não Funcionais

Descrevem como o sistema deve se comportar — qualidade, restrições técnicas, desempenho e atributos de qualidade que não estão diretamente ligados a funcionalidades específicas.

RNF-01 · DESEMPENHO

RNF-01.1

Tempo de resposta da busca por similaridade

should have



A busca completa — geração do embedding da foto enviada + consulta no ChromaDB + enriquecimento com dados do SQLite — deve retornar resultados em no máximo 8 segundos em hardware de desenvolvimento médio. A primeira inicialização pode levar mais tempo, pois o modelo CLIP é carregado em memória.

Meta	≤ 8 s por busca em hardware médio
Cold start	Até 60 s na primeira inicialização (carregamento do CLIP)
Após warm-up	Modelo em memória — buscas subsequentes mais rápidas

RNF-01.2

Tempo de cadastro de pet

should have



O cadastro de um pet com até 5 fotos deve ser concluído em no máximo 30 segundos. O tempo depende do número de fotos, pois cada uma gera um embedding individualmente. O app deve informar ao usuário que o processamento está em andamento.

Meta (1 foto)	≤ 8 s
Meta (5 fotos)	≤ 30 s
UX	Loading indicator ativo durante todo o processamento

RNF-02 · SEGURANÇA E PRIVACIDADE

RNF-02.1

Configuração de CORS no backend

must have ▼

O backend FastAPI deve configurar o middleware CORS para aceitar requisições do app Flutter. Para o ambiente de desenvolvimento/portfólio, a política `allow_origins=["*"]` é aceitável. Em produção real, seria restringido a origens específicas.

Middleware	<code>CORSMiddleware</code> do FastAPI
Dev	<code>allow_origins=["*"]</code>
iOS	Requer <code>NSAllowsArbitraryLoads</code> no <code>Info.plist</code> para HTTP local

RNF-02.2

Privacidade dos dados dos tutores

should have ▼

Os dados pessoais dos tutores (nome, telefone) devem ser coletados apenas para a finalidade de contato em caso de encontro do animal. O sistema não deve exibir informações de contato em listagens públicas gerais — apenas nos resultados de busca quando há similaridade relevante.

Armazenamento	SQLite local, sem transmissão para serviços externos
Exposição	Telefone visível apenas na tela de resultados de busca

RNF-03 · USABILIDADE

RNF-03.1

Interface responsiva para Android e iOS

must have ▼

O app Flutter deve funcionar corretamente em telas de diferentes tamanhos (5 a 6,7 polegadas), adaptando layout sem quebras visuais. O scroll deve ser suave e todos os elementos devem ser acessíveis por toque sem sobreposição.

Plataformas	Android 8.0+ · iOS 13+ (requer Mac para compilar)
Scroll	<code>SingleChildScrollView</code> em formulários longos
Toque	Área mínima de 44×44dp para elementos interativos

RNF-03.3

Documentação automática da API acessível

should have



O FastAPI deve expor a documentação interativa Swagger UI em /docs e ReDoc em /redoc, permitindo que qualquer membro do grupo teste os endpoints sem precisar do Postman ou do app Flutter.

Swagger	<code>http://localhost:8000/docs</code>
ReDoc	<code>http://localhost:8000/redoc</code>

RNF-04 · INFRAESTRUTURA E PORTABILIDADE

RNF-04.1

Ambiente completo via Docker Compose em um único comando

must have



Qualquer pessoa que clonar o repositório e tiver o Docker Desktop instalado deve conseguir subir o backend completo (FastAPI + ChromaDB + SQLite) com um único comando, sem instalar Python, bibliotecas ou configurar bancos de dados manualmente.

Comando	<code>docker compose up --build</code>
Critério	Funcionar em máquina limpa (Windows, macOS, Linux)
Tempo estimado	10-20 min (download do modelo CLIP na 1ª vez)

RNF-04.2

Persistência de dados entre reinicializações do container

must have



Os dados do sistema (imagens, banco SQLite e embeddings ChromaDB) devem persistir mesmo após parar e reiniciar os containers Docker. Isso é garantido por volumes nomeados declarados no `docker-compose.yml`.

Volume imagens	<code>petfinder_imagens</code> → <code>/backend/data/images</code>
Volume SQLite	<code>petfinder_sqlite</code> → <code>/backend/data/petfinder.db</code>
Volume ChromaDB	<code>petfinder_chroma</code> → <code>/backend/data/chroma</code>

RNF-04.3

Configuração de IP para diferentes ambientes de desenvolvimento

must have



O endereço base da API deve ser centralizado em um único arquivo de configuração no Flutter (`api_config.dart`), permitindo alternar entre emulador Android, dispositivo físico e simulador iOS sem alterar múltiplos arquivos.

Emulador Android	<code>http://10.0.2.2:8000</code>
------------------	-----------------------------------

RNF-05.1

Estrutura de repositório organizada e documentada

must have ▼

O repositório GitHub deve seguir uma estrutura de monorepo clara com separação entre /app (Flutter) e /backend (Python). O README deve conter demo visual, arquitetura, instruções de execução e justificativa técnica de cada tecnologia escolhida.

README

GIF ou vídeo demonstrativo obrigatório

Commits

Mensagens descritivas no padrão `feat:` , `fix:` , `docs:`

Branches

Trabalho em branches separadas; merge via Pull Request

RNF-05.2

Código comentado e legível para fins educacionais

must have ▼

Todo o código deve conter comentários explicativos em português, descrevendo a lógica de cada função e módulo. O objetivo é que qualquer membro do grupo consiga entender o código de outra área, mesmo sem conhecimento profundo da tecnologia. Isso é especialmente importante nos módulos de IA e no `docker-compose.yml`.

Python

Docstrings em todas as funções públicas

Dart

Comentários em blocos complexos e integrações HTTP

YAML

Comentários explicando cada chave do `compose`

3 Glossário

Definições dos termos técnicos usados neste documento.

Embedding

Vetor numérico de alta dimensão que representa o "significado" de uma imagem ou texto em forma matemática, permitindo comparação por similaridade.

CLIP

Modelo de IA da OpenAI que entende imagens e texto no mesmo espaço vetorial, permitindo comparar uma foto com uma descrição textual.

Similaridade cosseno

Métrica que mede o ângulo entre dois vetores. Vetores parecidos têm ângulo pequeno (similaridade próxima de 1);

ChromaDB

Banco de dados vetorial que armazena embeddings e realiza buscas por

diferentes têm ângulo grande (próximo de 0).

similaridade de forma eficiente, sem necessidade de servidor externo.

SQLite

Banco de dados relacional que funciona como um arquivo único em disco, sem necessidade de servidor. Ideal para dados convencionais (nome, telefone, cidade).

Docker volume

Mecanismo do Docker para persistir dados fora do ciclo de vida do container. Sem volumes, todos os dados são perdidos ao reiniciar o container.

Multipart/form-data

Formato HTTP para enviar texto e arquivos binários (imagens) na mesma requisição. Usado pelo Flutter para enviar foto + dados do pet juntos.

CORS

Mecanismo de segurança dos browsers e apps que bloqueia requisições entre origens diferentes. O FastAPI precisa configurá-lo para aceitar chamadas do Flutter.

Must have

Requisito obrigatório — o sistema não entrega valor sem ele. Deve estar presente na versão 1.0.

Should have

Requisito importante mas não crítico — agrega valor significativo. Implementar se houver tempo dentro dos 2 meses.